University of Waterloo

CS 398 Test 1

Spring 2026

Date:	Monday, June 29 2026
Exam period:	10:00 AM to 11:20 PM
Exam period:	11:30 AM to 12:50 PM
Duration:	80 minutes
Number pages:	12 (includes cover page)
Materials allowed:	No additional materials are allowed.

Additional instructions

1. Write your answers in the space provided. If you need more room, use the blank page near the end, and indicate this on the question page where you ran out of room.
2. There are a total of 24 possible points.
3. You may write in pen or pencil as long as your writing is legible. Your arguments must be logical, clear and easy to understand.
4. DO NOT write on the Crowdmark QR code at the top of the pages or your exam paper will not be scanned (and will receive a grade of zero).
5. The guideline reference is at the end of the exam.

Q1. (8 points) **Problem Statement:** You are a software engineer working on a campus event-planning app for student clubs. Your job is to produce a list of requirements for the app. You are given the following information:

Stakeholders: The complete stakeholder groups for this system are: club officers, club members, student attendees, guests, advisors, facilities staff, equipment staff, finance approvers, policy and compliance reviewers, accessibility staff, administrators, and system maintainers.

Draft requirements:

Club officers can create events with name, date, time, room preference, attendance estimate, setup needs, and equipment needs. Facilities staff must approve room bookings, but the system automatically confirms available rooms.

Club officers can request chairs, tables, microphones, projectors, and special equipment. Equipment staff review special requests, but requests submitted more than two weeks early are automatically approved.

Attendees and guests can RSVP with name, email, guest count, accessibility needs, and dietary restrictions. RSVP totals cannot exceed room capacity, but officers may manually add attendees after an event is full.

Public events appear publicly; private events do not. All student attendees can search every event by name.

Club officers can assign tasks to members. Members may accept or reject tasks, but required tasks cannot be rejected.

The system notifies relevant users of event changes, but avoids notifications to reduce email volume. Finance approvers review events near or over budget.

Advisors approve events before publication. Policy reviewers approve events with alcohol or outside vendors. Accessibility staff receive RSVP accessibility needs and arrange accommodations. Administrators manage club accounts and officer roles. System maintainers view logs and export audit records.

Required output format: Your team records every requirement in a fixed SRS template with these fields: ID, Description, Priority [High | Medium | Low], and Verification [Test | Inspection | Analysis | Demonstration].

Solution

Solution: The correct guidelines are Guideline 4 and Guideline 5.

Guideline 4: Provide a Structured Output Template is appropriate because the draft mixes many areas together (rooms, equipment, budgets,

RSVPs, tasks, approvals), so a required format keeps each requirement consistent and verifiable. To apply Guideline 4, I would prompt the LLM:

> Rewrite the requirements using this format, without adding features: Requirement ID:
> Description:
> Priority [High | Medium | Low]:
> Verification [Test | Inspection | Analysis | Demonstration]:

Guideline 5: Detect Conflicts and Undefined Terms is appropriate because the draft has vague terms ("available room," "special equipment," "near or over budget") and conflicts (facilities approval vs. auto-confirmation, RSVP limits vs. manual over-capacity adds, "search every event" vs. private visibility). To apply Guideline 5, I would prompt the LLM:

> Identify contradictions, vague terms, and conflicts using this format: Issue ID:
> Affected Requirement:
> Issue Type:
> Problem:
> Suggested Fix:

I would use the output as a revision checklist.

The other guidelines are less appropriate.
Guideline 1 is less appropriate because the stakeholders are already listed.
Guideline 2 is less appropriate because there is already a detailed draft, so the LLM does not need to interview the user from scratch.
Guideline 3 is less appropriate because the issue is not one vague statement, but a long draft needing organization and review.
Guideline 6 is less appropriate because the prompt is mainly about functional requirements, not non-functional requirements.

Q2. (8 points) At 08:52 AM Eastern, thirty-eight minutes before market open, a brokerage's payout worker is suspected of having replayed some ACH payouts after a timeout storm. The response team must build a **reviewable remediation utility** before treasury opens. A false positive would reverse legitimate customer funds. A false negative would leave duplicate payouts unrecovered and create an audit problem.

The treasury lead adds one hard constraint:

> If the meaning of "confirmed duplicate," "ambiguous," or "safe reversal" is inconsistent across our artifacts, you must surface that conflict before any classifier code is written. Do not bury policy conflicts inside implementation assumptions.

Staff note: the normalization fields are already trusted and out of scope. The fix will likely span parsing, evidence grouping, classification, and output emission. The danger is conflicting policy meaning across artifacts plus unsafe one-shot generation.

Treasury review gate: no classifier code will be reviewed until each output artifact has explicit preconditions, postconditions, and forbidden cases written down.

Starter material 1: existing parser stub

```
# replay_stub.py
from decimal import Decimal

def normalize(row):
    return {
        "payout_id": row["payout_id"],
        "customer_id": row["customer_id"],
        "amount": Decimal(row["amount"]),
        "trace_id": row["trace_id"] or None,
        "attempt": int(row["attempt"]),
        "status": row["status"],
        "bank_settlement_id": row["bank_settlement_id"] or None,
        "client_request_id": row["client_request_id"],
        "created_at": row["created_at"],
    }

def is_terminal(status):
    return status in {"SETTLED", "RETURNED", "REVERSED"}
```

Starter material 2: incident log excerpt

```
07:12:01 payout=P771 attempt=1 trace=T991 request=R44 submitted
07:12:03 payout=P771 attempt=1 trace=T991 request=R44
↪  timeout_waiting_for_bank_ack
07:12:08 payout=P771 attempt=2 trace=T991 request=R44 submitted
07:12:10 payout=P771 attempt=2 trace=T991 request=R44 settled
↪  bank_settlement=S201
07:12:11 payout=P771 attempt=1 trace=T991 request=R44
↪  settled_late_webhook bank_settlement=S201

07:14:14 payout=P772 attempt=1 trace=T992 request=R52 submitted
07:14:18 payout=P772 attempt=2 trace=T993 request=R52 submitted
07:14:25 payout=P772 attempt=2 trace=T993 request=R52 settled
↪  bank_settlement=S202

07:17:02 payout=P773 attempt=1 trace=T994 request=R61 submitted
07:17:09 payout=P773 attempt=1 trace=T994 request=R61 settled
↪  bank_settlement=S301
07:17:15 payout=P773 attempt=2 trace=T995 request=R61 submitted
07:17:22 payout=P773 attempt=2 trace=T995 request=R61 settled
↪  bank_settlement=S302
```

Starter material 3: structured case summaries

- `P771`: Shared payout intent `R44`, shared trace `T991`, and shared settlement `S201`. The retry and late webhook are still consistent with one actual money movement being observed twice.
- `P772`: Same payout intent `R52`, but the visible evidence only proves one settled money movement: attempt 2 settles with `S202`; attempt 1 does not independently settle.
- `P773`: Same payout intent `R61`, but two distinct bank settlement ids appear: `S301` and `S302`. This is the only case that clearly proves two settled movements.

Starter material 4: output contract and policy excerpts

- `confirmed_duplicates.csv`:
  Rows proven duplicated and safe for automated reversal consideration.
- `needs_manual_review.csv`:
  Ambiguous cases or cases with conflicting evidence.

- `safe_reversals.json`:
  Machine-readable reversal actions. Must never contain an ambiguous payout.
- Treasury policy memo: "Confirmed duplicate" means two settled money movements for the same payout intent, not merely two attempts.
- Ops runbook note: "Ambiguous" includes late webhooks, idempotent retries, or any case where one settlement could explain every observed event.
- Older analyst checklist: "Operations have historically placed multi-settlement cases in `confirmed_duplicates.csv` for review."

Business constraint: the utility must never auto-reverse a payout if the evidence could also be explained by a timeout, a late webhook, or an idempotent retry.

Solution: Full-credit set: Code Generation 1, Code Generation 4, and Code Generation 5.

Near-miss partial-credit set (cap 7/10): Code Generation 1, Code Generation 5, and Code Generation 6.

Why the full-credit set is best

Code Generation 1: Size each subtask so you can review every line is essential because the remediation utility spans parsing, evidence grouping, classification, and output emission. A one-shot script is unsafe in a money-moving incident. The work must be decomposed into reviewable units.

Code Generation 4: Drive generation with executable tests is necessary because the three concrete cases are exactly the kinds of evidence that should become tests before code. At minimum, the workflow should demand that `P771` stay manual because a late webhook explains the evidence, `P772` stay manual because the log does not prove two settled money movements, and `P773` become the positive duplicate case because it has two distinct settlement identifiers.

Code Generation 5: Specify preconditions, postconditions, and edge cases is also required because each output artifact needs a precise contract. The prompt should define when a payout counts as confirmed, when it must stay in manual review, and what preconditions must hold before any reversal action is emitted.

Why the near-miss set is not full credit

The near-miss set is disciplined and would still produce a careful workflow. It loses full credit because it replaces executable proof with self-critique. In this scenario, the named payout cases must become runnable classifier checks showing that `P771` and `P772` stay out of `safe_reversals.json` while `P773` is the only positive duplicate case.

Why other guidelines are weaker fits

Code Generation 2 is weaker because the problem is not selecting among alternative architectures. The dominant risk is policy inconsistency, not lack of design options.

Code Generation 3 is less central because the provided codebase context is intentionally thin and already trusted. The core challenge is evidence interpretation and output-contract safety, not deep integration with a large existing module.

Code Generation 6 is useful but weaker than **Code Generation 4** because self-critique does not replace executable proof on the named payout cases.

Interaction workflow

Initial prompt sequence

> Prompt 1: Before generating any code, break the remediation utility into reviewable subtasks and give the order in which they should be implemented. Do not emit a one-shot script.
>
> Prompt 2: Using the incident log, case summaries, output contract, and business constraint, write the exact preconditions, postconditions, and disallowed cases for the confirmed-duplicate file, the manual-review file, and the reversal-action output.
>
> Prompt 3: Before implementation, write executable tests for `P771`, `P772`, and `P773`, plus one extra adversarial case where the same client request id appears twice but only one settlement exists.
>
> Prompt 4: Now generate the utility in reviewable subtasks no longer than about 30 lines each. Do not emit a giant script.

Order of subtasks

- Normalize and parse evidence rows.

- Group events by payout intent and settlement evidence.
- Classify payouts against the resolved policy and test suite.
- Emit output files only after classifier tests pass.

Expected model output at each stage

- Stage 1: a reviewable implementation plan split into small subtasks.
- Stage 2: explicit contracts for each output file.
- Stage 3: executable tests for the three named cases and one adversarial case.
- Stage 4: small reviewable code units implementing the agreed classifier.

Review and validation

- Reject any workflow that asks for one-shot code instead of reviewable subtasks.
- Review each generated subtask line by line before requesting the next one.
- Run the named tests. Verify that `P771` and `P772` never enter `safe_reversals.json`.
- Check that the positive duplicate case requires two settled money movements, not merely two attempts.
- Reject any model output that substitutes self-critique for executable proof on the named payout cases.

Q3. (8 points) Problem Statement: You are a software engineer building a new scheduling feature, and you are close to a deadline. You need to collapse overlapping time ranges, and you find an existing utility, 'merge_intervals', in a large internal library you have never opened. Rather than read it closely, you ask your AI assistant to summarize it so you can decide whether to reuse it.

The assistant returns a polished, confident, professional summary: "Merges a list of intervals into a minimal set of non-overlapping intervals, returned sorted by start time. Raises a ValueError if the input list is empty. Does not modify the caller's input list. Safe to call concurrently from multiple threads." The tool reports 95% confidence. A teammate reviewing your pull request glances at the summary and says it "reads like real documentation, just reuse it and move on."

You have the function's source open in your editor and a working Python REPL right beside it and the exact setup that lets you check a claim in seconds rather than take it on trust.

Starter material:

````
```python
def merge_intervals(intervals, gap=0):
    ranges = intervals
    ranges.sort(key=lambda r: r[0])
    merged = []
    for start, end in ranges:
        if merged and start <= merged[-1][1] + gap:
            merged[-1] = (merged[-1][0], max(merged[-1][1], end))
        else:
            merged.append((start, end))
    return merged
```
````

Your task is to use an LLM to help you decide whether the summary is trustworthy enough to reuse this function in production without being swayed by how authoritative it sounds or how confident the tool claims to be, and without shipping on a statement that turns out to be false or incomplete.

Solution:

Which Guidelines Apply?

Guideline 4 (Do Not Trust Surface Fluency as a Quality Signal) is the first guideline you need. Everything pushing you toward accepting this summary

is a surface signal: it is fluent, it sounds authoritative, the tool reports 95% confidence, and a teammate wants to merge on the strength of how it reads. None of those is a safe proxy for usefulness – a summary can read like documentation and still omit a parameter, an error path, or a side effect that decides whether you can reuse the function. So you stop judging by how the summary sounds and judge instead by what it lets you do: the concrete task "decide whether I can safely reuse this in production." You re-anchor the summary to that task, discard the confidence language, reduce the prose to a list of concrete claims, and compare that list against the actual signature for omissions. That comparison is what surfaces the unmentioned ‘gap‘ parameter – a real part of the interface the polished summary silently dropped. This guideline catches the failure mode where the summary is true-sounding but incomplete or shaped for the wrong task.

Guideline 3 (Verify the Summary Against Behaviour, Not Just Plausibility) is the second guideline you need. Once Guideline 4 has flattened the summary into concrete claims – returns sorted by start, raises on empty input, does not mutate the input, thread-safe – each one is a testable proposition, and roughly one in five such AI-generated claims is demonstrably wrong even for strong models. Crucially, you cannot settle these by reading alone: the "does not modify the input" claim hinges on the single line ‘ranges = intervals‘, which looks like it makes a working copy but is actually an alias, so the in-place ‘ranges.sort()‘ quietly mutates the caller's list – exactly the kind of detail a confident summary makes you skim past. With the source and a REPL open, you convert each claim into a check and run it: an assertion for the ordering claim, a call on ‘[]‘ for the error-case claim, and a before-and-after comparison of the caller's list for the no-mutation claim. You do not accept a claim because it sounds right.

Why each of the other four guidelines does not apply here

Guideline 1 (Specify the Summary Intent Before Asking) governs the moment you *author* a request – choosing what kind of summary and for which reader before you ask. Here the summary already exists, it is already the right kind for your decision, and your doubt is about whether its statements are true and complete, not about what kind of summary it is. There is no intent to select; you are evaluating a finished artifact, so Guideline 1 answers a question this task does not pose.

Guideline 2 (Surface the Semantic Facts the AI Cannot See) is also an authoring move: enriching your prompt with the repository name, function name, identifiers, and data-flow so the model produces a better-grounded summary. But you already hold both the summary and the source – it is open in front of you.

The lever for trusting the summary is running the code and comparing actual behaviour to the claims, not feeding the model more facts to regenerate a description. Surfacing semantic facts would improve a summary you are writing; it does nothing to verify one you have been handed.

Guideline 5 (Adapt Your Prompt to the Language and Paradigm) does not apply because 'merge_intervals' is an ordinary imperative Python function: a sort, a loop, and a list accumulator. Guideline 5 matters when the code is in a logic, functional, or otherwise non-mainstream paradigm (Prolog, pointfree Haskell, Erlang) where the model defaults to the wrong vocabulary and needs explicit paradigm framing. There is no paradigm mismatch here, so paradigm-specific framing would add nothing.

Guideline 6 (Seed Project Style with a Few Local Exemplars) is about making a *new* summary match an existing project's house style by few-shotting with same-project (code, summary) exemplars. Your problem is not whether the summary reads like the rest of the codebase; it is whether it is true and complete enough to build on. Style-matching would polish how the summary reads while leaving the very claims you are worried about unchecked, so it answers a different question.

How would you implement the two guidelines?

Step 1: Apply Guideline 4. Strip the polish and re-anchor to your task. Send this prompt to the LLM:

Ignore how the previous summary is phrased and ignore any confidence language. For the task 'deciding whether I can safely reuse this function in production,' list only the concrete, checkable claims it makes – parameters accepted, return value and its ordering, error/exception cases, side effects on inputs, and concurrency – as a short bullet list, one claim per line, with no adjectives. Then compare that list against this signature and tell me anything in the signature the summary never addresses: 'merge_intervals(intervals, gap=0)'.

You chose this strategy because a fluent paragraph hides omissions, whereas an atomic claim list checked against the signature exposes them. The model returns the four behavioural claims and flags that the summary says nothing about 'gap' – an omission concealed by the confident prose, and a strike against the summary before a single test is run. Any claim phrased with confidence words ("robustly", "safely") you rewrite as a plain assertion or drop. The output is a clean checklist plus a list of what the summary failed to mention.

Step 2 – Apply Guideline 3. Turn each load-bearing claim into something you

run. Send this prompt: "For each claim in this list, write the minimal Python assertion or REPL snippet I can run against ‘merge_intervals‘ to confirm or refute it, and state the exact input to pass. For 'does not modify the input,' show a before-and-after check of the caller's list. Show what ‘gap‘ does with a snippet. Flag any claim that cannot be settled by a quick run."

You chose this strategy because the guideline treats a summary as a set of hypotheses to execute rather than trust, and the model drafts the small tests reliably even when its prose was wrong. You run the snippets and compare actual to claimed. The results: sorted-by-start holds; empty input returns ‘[]‘ rather than raising ‘ValueError‘ (false); and the caller's list is reordered after the call because ‘ranges = intervals‘ aliases rather than copies (the "no mutation" claim is false). Two of four claims fail, on top of the ‘gap‘ omission. You treat each mismatch as evidence the summary was wrong, not a fluke, and you do not reuse the function on the summary's word – you wrap the call defensively or correct the summary first. The thread-safety claim cannot be settled by a quick run, so per the guideline you treat it with extra suspicion and decline to rely on it rather than assume it.